

TECHNICAL REFERENCE

ShaderShow Architecture

An Electron-based GLSL shader editor with Shadertoy compatibility, real-time WebGL2 rendering, multi-channel inputs, mixer compositing, and NDI output.

GENERATED · 2026-04-16

ShaderShow Architecture

ShaderShow is an Electron-based GLSL shader editor with Shadertoy compatibility. It renders shaders in real-time using WebGL2, supports multi-channel inputs (textures, video, camera, audio FFT, NDI), multi-shader compositing via a mixer, tiled displays, A/B crossfade previews, Three.js scene rendering, and output via NDI, Syphon, and H.265 recording. A web-based remote control interface allows mobile/tablet operation.

Table of Contents

- [Process Model](#)
 - [Main Process](#)
 - [Entry Point & Orchestration](#)
 - [IPC Registry](#)
 - [Managers](#)
 - [Protocol Implementations](#)
 - [Renderer Process](#)
 - [Entry Point](#)
 - [Core Layer](#)
 - [Renderers](#)
 - [IPC Layer](#)
 - [UI Modules](#)
 - [Grid System](#)
 - [Tile System](#)
 - [Fullscreen Process](#)
 - [Preload Scripts](#)
 - [Shared Modules](#)
 - [Utilities](#)
 - [Type Definitions](#)
 - [Data Flow & Key Patterns](#)
 - [Feature Map](#)
-

Process Model

ShaderShow runs as four distinct Electron processes:

PROCESS	ENTRY SOURCE	BUILT OUTPUT	ROLE
Main	<code>src/main/app.ts</code>	<code>dist/main/app.js</code>	Node.js backend: file I/O, menus, dialogs, NDI, Syphon, recording, remote server, Art-Net, AI
Renderer	<code>src/renderer/app.ts</code>	<code>dist/renderer/app.js</code>	Chromium window: UI, WebGL rendering, parameter controls, editor, grid
Fullscreen	<code>src/fullscreen/app.ts</code>	<code>dist/fullscreen/app.js</code>	Separate Chromium window: native-resolution output rendering
Preload	<code>src/preload/preload.ts</code>	<code>dist/preload/preload.js</code>	Context bridge exposing <code>electronAPI</code> to renderer

Communication between processes uses Electron IPC:

- **Renderer <-> Main:** `ipcRenderer.invoke` / `ipcMain.handle` for request/response; `ipcRenderer.send` / `ipcMain.on` for fire-and-forget
- **Main <-> Fullscreen:** Forwarded via `FullscreenRelay` (22+ one-way channels)
- **Renderer -> Fullscreen:** Indirectly through Main as relay

Main Process

Entry Point & Orchestration

`SRC/MAIN/APP.TS`

The application entry point. This file bootstraps the entire Main process:

- **CLI argument parsing:** Supports `--headless` (no GUI window), `--port <n>` (remote server port override), and `--display <n>` (target display for fullscreen).
- **GPU optimization:** Sets Chromium flags for Linux NVIDIA cards (`--ignore-gpu-blocklist` , `--enable-gpu-rasterization` , `--enable-zero-copy` , `--disable-gpu-sandbox`).
- **appDir resolution:** Resolves the application data directory. In development, uses the project root; in packaged builds, resolves relative to `process.resourcesPath` .
- **Manager instantiation:** Creates all manager instances in dependency order:
 1. `FileManager` (path resolution, file I/O)
 2. `SettingsManager` (persistent settings)
 3. `WindowManager` (BrowserWindow lifecycle)
 4. `NDIManager` (NDI send/receive)
 5. `SyphonManager` (macOS frame output)
 6. `RecordingManager` (H.265 recording)
 7. `MenuBuilder` (application menu)
 8. `ExportManager` (bundle export/import)

9. `RemoteManager` (web remote server)
 10. `ClaudeManager` (AI assistant)
 11. `FullscreenRelay` (IPC forwarding)
 12. `ArtNetManager` (DMX control)
 13. `IPCRegistry` (IPC handler registration)
- **File operation helpers:** Defines `newFile()`, `openFile()`, `saveFile()`, `loadTexture()`, `loadVideo()`, `clearChannel()` which orchestrate dialog windows and IPC communication.
 - **App lifecycle:** `app.whenReady()` triggers window creation, menu setup, IPC registration, and optionally starts the remote server and NDI in headless mode. `window-all-closed` quits on non-macOS. `before-quit` shuts down NDI, Syphon, Art-Net, recording, and remote server.

IPC Registry

`SRC/MAIN/IPC-REGISTRY.TS`

Central registration point for all IPC handlers between Main and Renderer processes. Contains two categories:

Fire-and-forget handlers (`ipcMain.on`, 28 channels):

- `save-content` — Writes shader source to disk
- `ndi-frame` — Receives raw pixel data from renderer for NDI/Syphon/recording output
- `set-channel-ndi` / `set-ndi-resolution` / `toggle-ndi` — NDI configuration
- `toggle-syphon` — Syphon output toggle
- `toggle-remote` / `set-remote-token` — Remote server control
- `toggle-artnet` — Art-Net receiver control
- `save-grid-state` / `save-presets` / `save-view-state` / `save-tile-state` — Persistence
- `open-fullscreen` / `open-tiled-fullscreen` / `close-fullscreen` — Fullscreen window control
- `set-fullscreen-display` — Move fullscreen to a specific monitor
- `toggle-recording` / `set-recording-resolution` — Recording control
- `save-settings` — Settings persistence
- `shader-update` / `time-sync` / `param-update` / `mixer-update` / `ab-update` / `fullscreen-state` / `post-process-update` / `tiling-update` / `tile-data-update` / `preset-sync` — Forwarded to fullscreen via relay

Request/response handlers (`ipcMain.handle`, 37+ channels):

- `get-default-shader` / `get-default-scene` — Returns template code (DEFAULT_SHADER / DEFAULT_SCENE constants defined in this file)
- `load-grid-state` / `load-presets` / `load-view-state` / `load-tile-state` — State loading
- `find-ndi-sources` — NDI source discovery
- `start-recording` / `stop-recording` — Recording lifecycle
- `get-settings` / `get-remote-status` / `get-artnet-status` / `get-ndi-status` — Status queries
- `claude-*` handlers — AI key management, model listing, prompt streaming
- `save-texture` / `load-textures` / `delete-texture` — Texture CRUD in `data/textures/`
- `save-media` / `load-media-library` / `delete-media` — Media library CRUD in `data/media/`
- `export-button-data` / `import-button-data` — Single-slot shader export/import (.shader format)

- `export-app-state` / `import-app-state` — Full app bundle export/import
- `get-displays` — Monitor enumeration for fullscreen targeting
- `read-shader-file` — Read shader source from grid slot file

The file also defines `DEFAULT_SHADER` (a Shadertoy-compatible fragment shader template with `iResolution`, `iTime`, `iMouse` uniforms) and `DEFAULT_SCENE` (a Three.js scene template with `setup/animate` functions and `@param` directives).

Managers

`SRC/MAIN/MANAGERS/WINDOW-MANAGER.TS`

Manages all `BrowserWindow` instances:

- **Main window** (`createWindow()`): 1400x900 default, context isolation enabled, loads `index.html` . Tracks `resize/move` for window restore. If `--headless` mode, window is never created.
- **Fullscreen window** (`createFullscreenWindow()`): Frameless, transparent, always-on-top, positioned to fill a specific display. On KDE Wayland, uses KWin scripting via D-Bus (`qdbus`) to place windows on the correct output since Electron's `setBounds()` is unreliable on Wayland. The KWin script moves windows matching the title "ShaderShow Fullscreen" to the target display, then optionally sets `keepAbove` , `noBorder` , and `fullScreen` properties.
- **Tiled fullscreen** (`openTiledFullscreen()`): Same as fullscreen but sends tile configuration after window creation.
- **Custom resolution dialog** (`showCustomResolutionDialog()`): Small modal window for entering custom NDI/recording resolution. Loads `dialog.html` with `preload-dialog.js` .
- **Texture creator dialog** (`showTextureCreatorDialog()`): Modal for creating procedural textures from shader code. Loads `texture-creator.html` with `preload-texture-dialog.js` .
- **Display management**: `getDisplays()` enumerates monitors via `screen.getAllDisplays()` . `setFullscreenDisplay()` moves the fullscreen window to a different monitor, re-applying KWin scripting on Wayland.

`SRC/MAIN/MANAGERS/FILE-MANAGER.TS`

Centralizes all file system operations and path management:

- **Canonical paths**: Resolves `dataDir` (`<appDir>/data/`), `shadersDir` (`data/shaders/`), and specific state files: `grid-state.json` , `presets.json` , `settings.json` , `view-state.json` , `tile-state.json` , `claude-key.json` .
- **Directory management**: `texturesDir` (`data/textures/`), `mediaDir` (`data/media/`). Ensures directories exist on construction.
- **Grid state loading** (`loadGridState()`): Handles three formats:
 1. **v2 tabbed format**: `{ version: 2, tabs: [...] }` with named tabs each containing slots
 2. **Legacy array format**: Flat array of slot objects, migrated to single "Default" tab
 3. **File-scan fallback**: If no JSON exists, scans `shaders/` directory for `button*.glsl` files and constructs grid state from filenames
- **Shader file I/O**: `readShaderFile()` , `saveShaderContent()` for individual grid slot files (`buttonN.glsl` or `sceneN.js`).

- **Media library:** `saveMedia()` copies files to `data/media/` with collision-safe naming (appends `-1`, `-2`, etc.), `loadMediaLibrary()` returns all files, `deleteMedia()` removes a file.
- **Texture management:** `saveTexture()`, `loadTextures()`, `deleteTexture()` for `data/textures/`.
- **Button data export/import:** `exportButtonData()` creates a `.shader` JSON file with code + params + presets. `importButtonData()` restores from this format.
- **State persistence:** `saveGridState()`, `savePresets()`, `saveViewState()`, `saveTileState()`, `saveSettings()` — all write JSON to their respective files.

SRC/MAIN/MANAGERS/SETTINGS-MANAGER.TS

Manages persistent application settings stored in `data/settings.json`:

- **NDI settings:** `ndiResolution` (string like "1920x1080"), `ndiFrameSkip` (frame decimation factor). Special sentinel values: `-1` triggers custom resolution dialog, `0` means "match preview canvas".
- **Recording settings:** `recordingResolution` (string).
- **Grid settings:** `gridSlotWidth` (pixel width of grid thumbnails).
- **Remote settings:** `remoteEnabled`, `remotePort`, `remoteToken`. Token is auto-generated via `crypto.randomBytes(32)` if absent.
- **Art-Net settings:** `artnetEnabled`, `artnetUniverse`, `artnetMappings` (array of DMX channel-to-parameter mappings).
- **AI settings:** provider, model, key references.
- **Merge-on-save:** `saveSettings()` reads the current file, merges with incoming data to preserve keys like `paramRanges` that are managed by other code paths, then writes back.

SRC/MAIN/MANAGERS/NDI-MANAGER.TS

NDI (Network Device Interface) input and output management:

- **NDI Sender:** Single `NDISender` instance for outputting frames. Resolution management with sentinel handling (`-1` = show custom dialog, `0` = match preview size from incoming frame). Pre-allocated flip buffers for vertical frame flipping — WebGL renders bottom-to-top but NDI expects top-to-bottom. The flip is done row-by-row using typed array copies.
- **NDI Receivers:** Array of 4 `NDIReceiver` instances, one per iChannel slot. Each receiver runs an async receive loop delivering frames to a callback that forwards pixel data to the renderer via IPC.
- **Source discovery:** `findSources()` delegates to `NDIReceiver.findSources()` for discovering available NDI sources on the network.
- **Channel management:** `connectChannel(channelIndex, sourceName)` starts a receiver for a specific channel. `disconnectChannel(channelIndex)` stops it. `disconnectAll()` tears down all receivers on quit.
- **Resolution switching:** `setResolution()` handles the sentinel values, creates a new sender at the requested resolution, or defers to WindowManager for the custom resolution dialog.

SRC/MAIN/MANAGERS/RECORDING-MANAGER.TS

H.265 video recording via FFmpeg child process:

- **FFmpeg binary:** Uses `ffmpeg-static` npm package to locate the FFmpeg binary. Falls back to system `ffmpeg` if the static binary is missing.

- **Encoding pipeline:** Spawns FFmpeg with `-f rawvideo -pix_fmt rgba` input (piped via stdin), outputs H.265 in MP4 container. On macOS uses `hevc_videotoolbox` (hardware encoder); on other platforms uses `libx265` (software encoder).
- **Frame input:** `sendFrame(buffer, width, height)` receives raw RGBA pixel data. Performs vertical flip (same row-swap pattern as NDI) using a pre-allocated flip buffer. Ensures even dimensions by rounding up (H.265 requirement). Writes flipped data to FFmpeg's stdin.
- **Backpressure handling:** If FFmpeg's stdin buffer is full (high-water mark), frames are silently dropped to prevent memory buildup.
- **Lifecycle:** `startRecording(width, height)` opens a save dialog, spawns FFmpeg, returns the output path. `stopRecording()` closes stdin, waits for FFmpeg to finish, resolves with the output path.

SRC/MAIN/MANAGERS/CLAUDE-MANAGER.TS

AI assistant integration supporting multiple providers:

- **Providers:** Anthropic (`api.anthropic.com/v1/messages`) and OpenRouter (`openrouter.ai/api/v1/chat/completions`). Each provider has different API formats, model naming, and SSE parsing.
- **Key management:** API keys stored in `data/claude-key.json` with structure `{ anthropic?: string, openRouter?: string }`. `setKey()`, `getKeyStatus()`, `removeKey()` for CRUD.
- **Model listing:** `listModels()` fetches available models from the active provider. For Anthropic, returns a hardcoded list. For OpenRouter, calls their `/api/v1/models` endpoint.
- **Streaming:** `streamPrompt()` sends a prompt with system context (current shader code, scene code, available uniforms) and streams the response back via SSE. Each SSE chunk is forwarded to the renderer via IPC callback. Supports Anthropic's `content_block_delta` events and OpenRouter's `choices[0].delta.content` format.
- **Provider/model switching:** `setProvider()`, `setModel()` update the active configuration and persist to settings.

SRC/MAIN/MANAGERS/MENU-BUILDER.TS

Application menu construction using Electron's `Menu` API:

- **Dependency injection:** Constructor receives all managers and file operation callbacks, keeping menu logic decoupled from business logic.
- **File menu:** Submenu for each of 4 channels (iChannel0-3), each offering: Load Texture, Load Video, Camera Input, Audio FFT, Clear Channel. Plus NDI source submenus populated dynamically from `ndi-manager.findSources()`.
- **Edit menu:** Standard cut/copy/paste/undo/redo accelerators.
- **View menu:** Toggle DevTools, reload, zoom controls.
- **Shader menu:** Compile Shader (Cmd+Enter), Play/Pause (Space), Open Fullscreen, Open Tiled Fullscreen, NDI Output toggle, NDI Resolution submenu (common resolutions + custom), Syphon Output toggle (macOS only), Start/Stop Recording, Recording Resolution submenu, Run Benchmark.
- **Dynamic NDI sources:** The NDI source submenu is rebuilt each time the menu is opened, calling `findSources()` to discover currently available NDI senders on the network.

SRC/MAIN/MANAGERS/REMOTE-MANAGER.TS

Manages the remote control web server lifecycle:

- **Server lifecycle:** `startServer(port, token)` creates a `RemoteServer` instance and starts it. `stopServer()` shuts it down. Exposes `getStatus()` for UI state queries.
- **Query/dispatch pattern:** The remote server needs to query renderer state (current shader, params, mixer state, thumbnails) and dispatch actions (load shader, change params). Since the remote server runs in Main but renderer state lives in the Renderer process, this uses a correlation-ID pattern: Main sends a query to Renderer via IPC, Renderer responds with the data, Main correlates by ID and returns to the HTTP/WebSocket client. Timeout of 3 seconds prevents hanging requests.
- **Display enumeration:** Provides display list to the remote UI for fullscreen targeting.
- **Preview frame capture:** Captures the current renderer canvas as JPEG for MJPEG streaming to remote clients.

SRC/MAIN/MANAGERS/EXPORT-MANAGER.TS

Full application state export and import as `.shadershow` bundle files:

- **Export** (`exportAppState()`): Recursively reads the entire `data/` directory. Text files (`.json` , `.glsl` , `.js` , `.frag` , `.vert`) are stored as UTF-8 strings; binary files (images, videos) are stored as base64. The resulting object is JSON-stringified and gzip-compressed before writing to disk.
- **Import** (`importAppState()`): Reads and decompresses a `.shadershow` file. Restores all files to `data/` , creating subdirectories as needed. Includes path traversal prevention (rejects paths containing `..`). After successful import, offers to restart the application via dialog.
- **Use case:** Allows sharing complete ShaderShow configurations (all shaders, grid state, presets, textures, media) as a single portable file.

SRC/MAIN/MANAGERS/FULLSCREEN-RELAY.TS

IPC message forwarding between Main, Renderer, and Fullscreen windows:

- **One-way forwarding channels** (22 channels): `shader-update` , `time-sync` , `param-update` , `mixer-update` , `mixer-channel-update` , `mixer-remove-channel` , `tile-data-update` , `tile-layout-update` , `ab-update` , `ab-set-side` , `ab-set-composition` , `ab-set-crossfade` , `post-process-update` , `tiling-update` , `fullscreen-state` , `set-render-mode` , `asset-update` , and more. Messages from the renderer are forwarded to fullscreen, and vice versa where appropriate.
- **Bidirectional preset-sync:** The `preset-sync` channel detects the sender (renderer vs fullscreen) and forwards to the other window, enabling preset changes from either side.
- **Special handling:** `fullscreen-state` messages trigger fullscreen window creation/destruction in WindowManager when the state toggles.
- **Registration:** `registerAll()` sets up all listeners. `unregisterAll()` removes them on shutdown.

SRC/MAIN/MANAGERS/SYPHON-MANAGER.TS

macOS-only Syphon frame output:

- **Syphon protocol:** Uses `node-syphon` npm package which provides either a Metal-based or OpenGL-based Syphon server depending on the macOS version.
- **Frame sending:** `sendFrame(buffer, width, height)` performs the same vertical flip as NDI (pre-allocated flip buffer, row-by-row swap) then passes the flipped RGBA data to the Syphon server.
- **Lifecycle:** `start(name, width, height)` creates a `SyphonSender` with the given server name. `stop()` destroys it. `isActive()` reports current state.

- **Error handling:** Catches import errors gracefully since `node-syphon` is macOS-only and may not be available.

`SRC/MAIN/MANAGERS/ARTNET-MANAGER.TS`

Art-Net (DMX over Ethernet) receiver for external lighting/VJ controller integration:

- **Protocol:** Listens for ArtDmx packets on UDP port 6454 (standard Art-Net port). Parses the Art-Net header to extract universe number, sequence, and 512-byte DMX data frame.
- **Universe filtering:** Only processes packets matching the configured universe number.
- **DMX frame diffing:** Maintains previous DMX frame buffer. On each packet, compares channel-by-channel and only emits changes for channels that differ from the previous frame. This prevents flooding the renderer with redundant updates.
- **Mapping system:** Each mapping links a DMX channel (1-512) to a target:
 - `param` — Maps DMX value (0-255) to a shader parameter's min-max range
 - `speed` — Maps to playback speed
 - `vp-recall` — Rising-edge trigger: recalls a visual preset when DMX value crosses threshold
 - `preset-recall` — Rising-edge trigger: recalls a local parameter preset
 - `blackout` — Rising-edge trigger: toggles blackout state
 - `mixer-select` — Maps DMX value ranges to mixer channel selection
- **Rising-edge detection:** For trigger-type mappings, tracks per-channel "was-above-threshold" state to fire only on the transition from below to above threshold (prevents repeated triggering while a fader is held high).
- **Batched IPC:** Accumulates changes during a single packet parse, then emits a single `artnet-dmx-update` IPC message with all changes. Limits emission rate to ~30fps.

Protocol Implementations

`SRC/MAIN/NDI-SENDER.TS`

Low-level NDI sender wrapper:

- **Platform detection:** Requires `grandiose-mac` on macOS or `grandiose` on other platforms. Both provide the same API but differ in native bindings.
- **NDISender class:** Wraps `grandiose.send()`. Constructor takes a sender name. `send(buffer, width, height)` constructs an NDI frame object with RGBA FourCC type, progressive frame format, and high-resolution timestamps via `process.hrtime()`.
- **Timestamp calculation:** Converts `hrtime` nanoseconds to NDI's 10MHz clock units (100ns ticks) for accurate frame timing.
- **Destruction:** `destroy()` releases the native NDI sender resource.

`SRC/MAIN/NDI-RECEIVER.TS`

Low-level NDI receiver wrapper:

- **NDIReceiver class:** Creates an NDI receiver bound to a specific source by name. Runs an async `receiveLoop()` that continuously calls `grandiose.receive()` with a timeout.
- **Static `findSources()`:** Uses `grandiose.find()` to discover NDI senders on the local network. Returns array of source objects with name and URL.

- **Frame delivery:** Each received frame's data buffer is passed to a user-provided callback, typically forwarding to the renderer via IPC as `ndi-input-frame`.
- **Timeout handling:** `receive()` may time out if no frame is available; the loop simply retries. This prevents blocking indefinitely on a stalled source.
- **Lifecycle:** `stop()` sets a flag that exits the receive loop on next iteration, then calls `destroy()` on the native receiver.

SRC/MAIN/SYPHON-SENDER.TS

Low-level Syphon sender wrapper (macOS only):

- **SyphonSender class:** Wraps `node-syphon`'s `SyphonServer` (Metal or OpenGL variant depending on macOS version).
- **Constructor:** Takes a server name (e.g., "ShaderShow") and initial dimensions.
- `publishFrame(buffer, width, height)`: Sends raw RGBA pixel data to Syphon clients. The frame is published at the given dimensions.
- **Error isolation:** Import failure is caught silently since this module is platform-specific.

SRC/MAIN/REMOTE-SERVER.TS

Express + WebSocket server for web-based remote control:

- **Express app:** Serves static files from `remote/` directory (the web remote UI). All `/api/*` routes require token authentication via `Authorization: Bearer <token>` header or `?token=<token>` query parameter.
- **REST API routes:**
 - `GET /api/state` — Full app state (grid, mixer, params, settings)
 - `GET /api/thumbnails` — Grid slot thumbnails as base64 JPEG (LRU cache of 200 entries)
 - `GET /api/preview` — Single JPEG frame capture from renderer
 - `GET /api/preview/stream` — MJPEG stream at ~10fps for live preview
 - `GET /api/displays` — Monitor enumeration
 - `POST /api/action` — Dispatch actions (load-shader, set-param, mixer-select, etc.)
- **WebSocket:** Accepts upgrade on `/ws` path. Token validated on connection. Incoming messages are routed through `WS_ACTIONS` map which supports the same actions as the REST endpoint but with lower latency. State change notifications are broadcast to all connected clients.
- **MJPEG streaming:** Maintains a list of active response streams. Periodically captures preview frames from the renderer and writes multipart JPEG boundaries to all connected streams. Clients see a continuous video feed in an `<img>` tag.
- **Thumbnail cache:** LRU cache (max 200 entries) keyed by slot index + content hash. Prevents re-encoding thumbnails that haven't changed.

Renderer Process

Renderer Entry Point

SRC/RENDERER/APP.TS

The renderer's initialization sequence, executed on `DOMContentLoaded`. Modules are initialized in a specific order due to dependencies:

1. `initConsolePanel()` — Message log panel (needed early for error display)
2. `initEditor()` — Ace code editor with GLSL mode
3. `initRenderer()` — WebGL2 context and ShaderRenderer on `#shader-canvas`
4. `initControls()` — Toolbar buttons, resizer panels
5. `initSettings()` — Settings dialog
6. `initParams()` — Parameter slider UI
7. `initPostProcess()` — Post-processing sliders
8. `initTiling()` — Tiling repetition controls
9. `initMouseAssignment()` — Mouse-to-parameter mapping
10. `initPresets()` — Local preset management
11. `initResizer()` — Panel resize handles
12. `registerIPCHandlers()` — All main→renderer IPC listeners
13. `initMixer()` — Mixer channel bar
14. `initTileConfig()` — Tile display configuration
15. `compileShader()` — Initial shader compilation
16. `initShaderGrid()` — Grid slot creation and event binding
17. `restoreViewState()` — Restore saved panel sizes and visibility
18. `cacheElements()` — Cache frequently accessed DOM elements for render loop
19. `startRenderLoop()` — Begin `requestAnimationFrame` loop

Core Layer

`SRC/RENDERER/CORE/STATE.TS`

The global mutable state singleton that all renderer modules import and modify directly:

- **Editor state:** `editor` (Ace editor instance), `currentSlot` (active grid slot index), `shaderDirty` flag.
- **Renderer references:** `renderer` (active IRenderer), `shaderRenderer` (ShaderRenderer), `sceneRenderer` (ThreeSceneRenderer), `renderMode` ('shader' | 'scene' | 'asset').
- **Channel state:** `channels` array of 4 entries, each with `type` (ChannelType), `source` (URL/name), `texture` (WebGLTexture).
- **Mixer state:** `mixerChannels` array (up to `MAX_MIXER_CHANNELS = 8`), each with `slotIndex`, `alpha`, `blendMode`, `renderMode`, `code`, `params`. `mixerEnabled` flag, `mixerActiveChannel` index.
- **Visual presets:** `visualPresetTabs` (tabbed groups), `activeVPTab` index.
- **Tiled preview:** `tiledPreviewEnabled`, `tiledPreviewBounds` (cached tile pixel positions), `tileData` and `tileLayout` configuration.
- **A/B crossfade:** `abEnabled`, `abSideA` / `abSideB` (each with mode, slotIndex, code, params), `abCrossfade` (0.0-1.0).
- **Output state:** `ndiEnabled`, `syphonEnabled`, `recording`, with per-channel frame skip counters (`ndiFrameSkip`, `syphonFrameSkip`, `recordingFrameSkip`) and frame counters for decimation.

- **Fullscreen tracking:** `fullscreenEnabled` , `fullscreenAdaptiveSkip` for adaptive frame rate limiting when fullscreen is active.
- **Remote state:** `remoteEnabled` with debounced `notifyRemoteStateChange()` that batches state notifications to the remote server (150ms debounce).

SRC/RENDERER/CORE/RENDER-LOOP.TS

The main `requestAnimationFrame` rendering loop:

- **Render priority:** The loop checks modes in priority order:
 1. **A/B mode:** If `abEnabled` , renders both A and B sides using their respective renderers, composites with crossfade alpha on a 2D canvas overlay.
 2. **Tiled mode:** If `tiledPreviewEnabled` , renders all tile slots using shared `MiniShaderRenderer` instances into a grid layout on the preview canvas.
 3. **Mixer mode:** If `mixerEnabled` , renders all active mixer channels via `MiniShaderRenderer` instances, composites them with alpha/blend on the 2D overlay canvas.
 4. **Asset mode:** If `renderMode === 'asset'` , delegates to `AssetRenderer` .
 5. **Normal mode:** Renders the current shader or scene via the active `IRenderer` .
- **Stats display:** DOM updates for FPS, frame time, resolution, and render mode are throttled to every 250ms to avoid layout thrashing.
- **Tiled preview:** Uses `state.tiledPreviewBounds` (pre-calculated pixel positions from `calculateTileBounds()`) for each tile. A shared `MiniShaderRenderer` is resized via `ensureSharedCanvasSize()` once per frame, then `renderDirect()` draws each tile to the correct canvas position.
- **Output frame sending:** After rendering, calls `sendOutputFrames()` from `frame-sender.ts` . Applies per-channel frame skip counters (e.g., NDI sends every Nth frame). The adaptive skip counter increases when fullscreen is active to reduce main-window overhead.
- **Tile click handling:** Listens for clicks on the preview canvas, maps pixel coordinates to tile indices using the cached bounds, and updates the selected tile with param UI refresh.

SRC/RENDERER/CORE/RENDERER-MANAGER.TS

Renderer lifecycle management:

- `initRenderer()` : Creates a `ShaderRenderer` instance on the `#shader-canvas` DOM element. Stores it in `state.shaderRenderer` and `state.renderer` .
- `restartRender()` : Destroys the current renderer's GL state and reinitializes it. Used when switching modes or recovering from GL context loss.
- `ensureSceneRenderer()` : Lazy-loads the `ThreeSceneRenderer` . Since Three.js and Babel are large dependencies, they're only loaded when a scene is first opened. Returns a promise that resolves when the scene renderer is ready.
- `setRenderMode(mode)` : Switches between 'shader', 'scene', and 'asset' modes. Reinitializes GL state as needed since Three.js and raw WebGL2 have conflicting GL state expectations.
- `detectRenderMode(code)` : Content-based detection — if code contains `function setup()` and `function animate()` , it's a scene; otherwise it's a shader.

Renderers

[SRC/RENDERER/RENDERERS/SHADER-RENDERER.TS](#)

The primary WebGL2 rendering pipeline (~900 lines). This is the core of ShaderShow:

- **Shadertoy compatibility:** Implements all standard Shadertoy uniforms:
 - `iResolution` (vec3) — Canvas width, height, pixel ratio
 - `iTime` (float) — Elapsed time in seconds
 - `iTimeDelta` (float) — Time since last frame
 - `iFrame` (int) — Frame counter
 - `iMouse` (vec4) — Mouse position (xy = current when pressed, zw = click position)
 - `iDate` (vec4) — Year, month, day, seconds since midnight
 - `iChannel0-3` (sampler2D) — Input textures
 - `iChannelResolution[4]` (vec3) — Per-channel texture dimensions
- **Custom uniforms via `@param`:** After compilation, parses the shader source for `@param` directives (via `parseShaderParams()`). Creates uniform locations for each custom parameter and updates them each frame from the current parameter values.
- **Channel texture system:** 4 channel slots, each supporting:
 - **Image:** Static texture loaded from file, applied once
 - **Video:** `<video>` element, texture updated every frame via `texImage2D`
 - **Camera:** MediaStream from `getUserMedia()`, rendered through hidden `<video>` element
 - **Audio FFT:** `AnalyserNode` from Web Audio API, frequency data written to texture as a 512x2 luminance texture (row 0 = frequency, row 1 = waveform)
 - **NDI:** Frames received from Main process via IPC, uploaded as RGBA texture
 - **Shader-generated** (`ShaderTextureChannel`): Renders a secondary shader to FBO, output texture used as input
- **Beat detection:** Optional `BeatDetector` instance fed from audio channel data. Exports `iBassLevel`, `iMidLevel`, `iHighLevel`, `iBPM` uniforms for sound-reactive shaders.
- **Post-processing uniforms:** `iLuminance`, `iHue`, `iSaturation`, `iContrast` from the post-processing panel.
- **Tiling uniforms:** `iTilingCols`, `iTilingRows`, `iTilingSpacing`, `iTilingBgColor` for shader-level repetition.
- **Compile cycle:** `compile(fragmentSource)` wraps the user's GLSL in a standard preamble (uniforms + `mainImage` to `main` adapter via `buildFragmentWrapper()`), creates program, caches all uniform locations. Returns `CompileResult` with success/error info including line numbers.
- **Render cycle:** `render(time, deltaTime)` sets the viewport, binds the program, updates all uniforms (standard + custom + post-process + tiling), binds channel textures, updates video/camera/audio textures, draws a fullscreen quad.
- **Mouse handling:** Tracks mouse position relative to canvas, with separate tracking for mouse-down position (Shadertoy convention where `iMouse.zw` stores the click-start position with sign encoding for button state).

[SRC/RENDERER/RENDERERS/MINI-SHADER-RENDERER.TS](#)

Lightweight renderer for grid thumbnails and mixer/tile compositing:

- **Shared WebGL2 context:** All `MiniShaderRenderer` instances share a single offscreen `<canvas>` with one WebGL2 context. This avoids the browser's limit on concurrent WebGL contexts (typically 8-16). The shared canvas is created on first instantiation.
- **Per-instance state:** Each instance has its own compiled program, uniform locations, parameter values, and channel textures. Switching between instances requires rebinding the program and uniforms.
- `renderDirect(ctx2d, x, y, width, height)` : Renders the shader on the shared WebGL canvas, then draws the result onto any 2D canvas context at the specified position and size using `drawImage()` . This enables compositing multiple shaders onto a single visible canvas.
- `ensureSharedCanvasSize(width, height)` : Static method that resizes the shared offscreen canvas if the requested dimensions exceed the current size. Called once per frame before rendering all thumbnails/tiles.
- **Compilation:** Same `buildFragmentWrapper()` + `compileProgram()` pipeline as `ShaderRenderer`, but with fewer uniforms (no mouse, no beat detection). Supports custom `@param` uniforms.
- **Channel support:** Supports image, video, and shader-generated texture channels. NDI and camera channels are not supported in mini renderers.
- **Texture lifecycle:** `setFileTexture(channel, dataUrl)` loads a texture from base64 data URL. `setShaderTexture(channel, glslCode, resolution)` creates a `ShaderTextureChannel` for that channel.

SRC/RENDERER/RENDERERS/THREE-SCENE-RENDERER.TS

Three.js scene renderer implementing the `IRenderer` interface:

- **Scene model:** Users write JavaScript/TypeScript code with `setup()` and `animate()` functions. `setup(THREE, scene, camera)` initializes the 3D scene. `animate(time, deltaTime, params)` is called each frame for animation.
- **JSX compilation:** If scene code contains JSX syntax, Babel transforms it before evaluation. This enables a React-like declarative syntax for building Three.js scenes.
- **@param support:** Parses the same `@param` directives as `ShaderRenderer`. Parameter values are passed to the `animate()` function as a key-value object.
- **Compilation:** `compile(source)` evaluates the user code in a sandboxed function scope, extracts `setup` and `animate` functions, creates a new Three.js `Scene` and `PerspectiveCamera` , calls `setup()` . Returns `CompileResult` .
- **Rendering:** `render(time, deltaTime)` calls the user's `animate()` function, then `threeRenderer.render(scene, camera)` . The Three.js `WebGLRenderer` shares the same canvas as `ShaderRenderer` (obtained via `renderer.domElement`).
- **Resource cleanup:** `dispose()` traverses the scene graph, disposing geometries, materials, and textures to prevent GPU memory leaks.

SRC/RENDERER/RENDERERS/TILE-RENDERER.TS

Viewport-scissored renderer for the tiled fullscreen display:

- **Shared GL context:** All `TileRenderer` instances in a tiled fullscreen share a single WebGL2 context from the fullscreen canvas. Each tile renders to a specific viewport region using `gl.viewport()` and `gl.scissor()` .
- **TileSharedState** : Interface for state shared across all tiles in a tiled display: `time` , `deltaTime` , `frame` , `mouse` , `channels` , `postProcess` , `tiling` . This avoids duplicating global state per tile.

- **Per-tile state:** Each tile has its own compiled program, uniforms, parameters, and optionally its own channel textures (file textures and shader textures are per-tile; video/camera/audio/NDI are shared).
- **Compilation:** `compile(source)` uses the same fragment wrapper and program compilation as `ShaderRenderer`.
- **Rendering:** `render(x, y, width, height, shared)` sets viewport and scissor to the tile's bounds, binds the program, updates uniforms from shared state + per-tile params, draws a fullscreen quad. The scissor test ensures each tile only affects its designated pixels.
- **Channel textures:** `setFileTexture()` for per-tile image textures. `setShaderTexture()` for per-tile shader-generated textures. Shared channels (camera, audio, video) are passed via `TileSharedState`.

SRC/RENDERER/RENDERERS/BEAT-DETECTOR.TS

Energy-based BPM detection for sound-reactive shaders:

- **Algorithm:** Maintains a rolling buffer of audio energy values (from FFT frequency bins). Computes a dynamic threshold as `mean + k * stddev` of the energy buffer. When current energy exceeds the threshold, a beat is detected.
- **Configurable frequency bins:** Splits the FFT spectrum into bass (20-250Hz), mid (250-2000Hz), and high (2000-20000Hz) bands. Each band's energy is computed separately.
- **BPM calculation:** Tracks beat timestamps in a rolling window. BPM is derived from the median inter-beat interval.
- **Exported values:** `bassLevel`, `midLevel`, `highLevel` (0.0-1.0 normalized energy), `bpm` (beats per minute). These are read by `ShaderRenderer` and exposed as `iBassLevel`, `iMidLevel`, `iHighLevel`, `iBPM` uniforms.
- **Smoothing:** Energy levels are smoothed with an exponential moving average to prevent jittery values.

SRC/RENDERER/RENDERERS/ASSET-RENDERER.TS

Image and video asset renderer for asset grid slots:

- **Rendering:** Draws image or video content onto the WebGL canvas with configurable transformations:
 - **Crop:** x, y, width, height (normalized 0-1)
 - **Scale:** Uniform scale factor
 - **Keep aspect ratio:** Boolean, preserves source aspect ratio within canvas
 - **Repeat:** Integer, tiles the asset in a grid pattern
 - **Scroll speed:** Horizontal and vertical scroll (pixels per second)
- **Parameter definitions:** `ASSET_PARAM_DEFS` (crop, scale, aspect, repeat) and `VIDEO_PARAM_DEFS` (adds playback rate, loop, scroll speed). These are used by the parameter panel to generate appropriate UI controls.
- **Video handling:** Uses a `<video>` element for video assets. Supports play/pause, loop, and playback rate. Texture updated each frame from the video element.

SRC/RENDERER/RENDERERS/GL-UTILS.TS

Shared WebGL2 utility functions and types used by all renderers:

- **Types:**
 - `StandardUniforms` — Cached uniform locations for all Shadertoy-compatible uniforms
 - `CustomParamUniforms` — Map of parameter name to uniform location

- `TextureInfo` — Channel texture state (WebGLTexture, type, dimensions, video element)
- `VERTEX_SHADER_SOURCE` : The fullscreen quad vertex shader used by all renderers. Draws a screen-filling triangle pair with UV coordinates.
- `setupFullscreenQuad(gl)` : Creates and binds a VAO with a position buffer for the fullscreen quad. Returns the VAO for later binding.
- `buildFragmentWrapper(userCode, params, textureDirectives)` : Wraps user-authored GLSL in a standard preamble containing:
 - All Shadertoy uniform declarations
 - Custom `@param` uniform declarations (generated from ParamDef array)
 - `@texture` sampler2D declarations (for shader-generated textures)
 - Post-processing uniforms (luminance, hue, saturation, contrast)
 - Tiling uniforms (cols, rows, spacing, bgColor)
 - Beat detection uniforms (bass, mid, high levels, BPM)
 - A `void main()` that calls the user's `void mainImage(out vec4, in vec2)` with proper coordinate setup
- `compileProgram(gl, vertexSource, fragmentSource)` : Compiles and links a WebGL program. Returns the program or error string with line numbers for the shader compiler error log.
- `cacheStandardUniforms(gl, program)` : Looks up all standard uniform locations and returns a `StandardUniforms` object for efficient per-frame updates.
- `cacheCustomParamUniforms(gl, program, params)` : Looks up uniform locations for all custom `@param` parameters.
- `setCustomUniforms(gl, paramUniforms, params, paramValues)` : Sets uniform values for all custom parameters, handling float, int, bool, vec2/3/4, and color types.
- `loadTextureFromDataUrl(gl, dataUrl)` : Creates a WebGL texture from a base64 data URL. Returns a promise resolving to `{ texture, width, height }`.
- `createBuiltinTexture(gl, name)` : Creates procedural textures (e.g., noise, checkerboard) by name.

`SRC/RENDERER/RENDERERS/SHADER-TEXTURE-CHANNEL.TS`

Framebuffer-based render-to-texture for `@texture shader()` directives:

- **Purpose:** When a shader source contains `@texture iChannelN shader(glslCode) [resolution]`, this class renders the specified GLSL code to a framebuffer object (FBO) and uses the output texture as an input channel.
- **FBO setup:** Creates a framebuffer with a color attachment texture. The texture dimensions come from the `@texture` directive's resolution parameter.
- **Resolution modes:** If resolution values are 0-1, they're treated as fractions of the main canvas size. Values >1 are treated as absolute pixel dimensions.
- **Own program:** Each ShaderTextureChannel has its own compiled shader program and uniform set (standard Shadertoy uniforms + any `@param` uniforms in the texture shader).
- **Render cycle:** `render(time, deltaTime, frame)` binds the FBO, sets viewport, renders the texture shader, unbinds. The output `texture` property is then bound to the appropriate `iChannel` slot by the parent renderer.
- **Lifecycle:** `compile(glslCode)` compiles the texture shader. `dispose()` cleans up the FBO, texture, and program.

Renderer IPC Layer

SRC/RENDERER/IPC/IPC-HANDLERS.TS

Registers all IPC listeners for messages from the Main process to the Renderer:

- **File operations:** `file-opened` (loads shader/scene code into editor + grid slot), `new-file` (resets to default shader), `texture-loaded` (sets channel texture from file path), `video-loaded` (sets channel to video playback).
- **Channel setup:** `camera-requested` (starts `getUserMedia` and assigns to channel), `audio-requested` (starts `AudioContext` + `AnalyserNode` for FFT), `ndi-input-frame` (uploads NDI pixel data as channel texture).
- **Settings:** `settings-changed` (updates state from settings, re-applies NDI frame skip, etc.).
- **Compilation:** `compile-shader` (triggers shader recompilation from menu/shortcut).
- **Playback:** `toggle-playback` (play/pause the render loop time).
- **Recording:** `recording-status` (updates UI to reflect recording state).
- **Fullscreen:** `fullscreen-opened` / `fullscreen-closed` (tracks fullscreen state for adaptive frame skipping).
- **Remote control queries:** `remote-query-state` (serializes full renderer state for remote clients), `remote-query-thumbnails` (captures grid thumbnails as base64 JPEG), `remote-query-preview` (captures preview canvas as JPEG).
- **Remote control actions:** `remote-action` (dispatches actions like shader loading, param changes, mixer selection, triggered from the web remote).
- **Art-Net:** `artnet-dmx-update` (applies DMX value changes to mapped parameters/triggers).
- **Benchmark:** `run-benchmark` (triggers GPU/CPU benchmark).

SRC/RENDERER/IPC/FRAME-SENDER.TS

Unified frame capture and distribution for NDI, Syphon, and recording outputs:

- `readCanvasPixels()` : Single `gl.readPixels()` call captures the current canvas content as RGBA. If A/B mode is active, composites both sides onto a temporary canvas before reading. Returns a reusable `Uint8Array` buffer (pre-allocated per output channel to avoid GC pressure).
- `sendOutputFrames()` : Called once per render frame. Checks which outputs are active (NDI, Syphon, recording) and applies per-channel frame skip counters. If any output needs a frame this tick, calls `readCanvasPixels()` once and sends the same buffer to all active outputs via IPC (`ndi-frame` channel with a `targets` field).
- **Frame skip logic:** Each output has a skip counter (e.g., `state.ndiFrameSkip = 2` means send every 2nd frame). The counter is maintained in `state` and compared against a per-output frame counter that increments each render frame.
- **Recording toggle:** `startRecording()` invokes the main process to start FFmpeg, optionally switching to the recording resolution. `stopRecording()` stops FFmpeg and restores the previous resolution.

UI Modules

SRC/RENDERER/UI/EDITOR.TS

Ace code editor integration:

- **Setup:** Initializes Ace editor on `#editor` DOM element with GLSL syntax mode, dark theme, and standard editor features (line numbers, bracket matching, minimap).
- **Tabbed editing:** Supports multiple open files as editor tabs. Each tab tracks its own content, cursor position, scroll state, and undo history. Tab switching saves/restores these per-tab states.
- **Auto-compile:** 500ms debounce on content changes triggers shader compilation. The debounce timer resets on each keystroke, so rapid typing doesn't cause excessive recompilation.
- **Compile with directives:** Before compilation, pre-processes the shader source:
 1. Extracts `@texture` directives and creates/updates `ShaderTextureChannel` instances
 2. Extracts `@param` directives for uniform generation
 3. Passes processed source to the active renderer's `compile()` method
- **Error display:** Compilation errors are shown as Ace annotations (red markers in the gutter) with line numbers mapped from the wrapped shader back to user code.

SRC/RENDERER/UI/CONTROLS.TS

Toolbar and panel controls:

- **Playback:** Play/pause button toggles `state.playing` and updates the render loop. Reset time button sets `state.time = 0`.
- **Resolution dropdown:** Preset resolutions (360p, 480p, 720p, 1080p, etc.) plus custom. Resizing the canvas triggers GL viewport update.
- **Fullscreen display select:** Dropdown populated from `getDisplays()` IPC call. Selecting a display calls `open-fullscreen` with the display index.
- **Resizer panels:** Draggable splitters between:
 - Editor panel and preview panel (horizontal)
 - Top panels and bottom panels (vertical)
 - Grid area and parameter area (bottom horizontal)
 - Main content and visual presets sidebar (right edge) These store positions in view-state for persistence.
- **Time sync:** When fullscreen is active, periodically sends `time-sync` to keep fullscreen renderer's clock aligned with the main window.

SRC/RENDERER/UI/PARAMS.TS

Custom shader parameter UI generation:

- **@param parsing:** After shader compilation, `updateParamsUI()` reads the `@param` definitions from the compiled shader and generates appropriate HTML controls:
 - `float` → Range slider with min/max/step and numeric display
 - `int` → Integer slider
 - `bool` → Checkbox
 - `color` → Color picker (RGB hex input)
 - `vec2` → Two linked sliders (x, y)
 - `vec3` → Three sliders or color picker (if named with "color")
 - `vec4` → Four sliders

- **Value binding:** Slider changes update `state.paramValues[name]` which the renderer reads each frame for uniform updates. Bidirectional — loading a preset updates both the state and the slider positions.
- **Mouse assignment:** Parameters can be mapped to mouse X/Y axes for real-time control. `initMouseAssignment()` sets up a mode where clicking a parameter assigns it to a mouse axis.
- **Tile parameter routing:** When a tile is selected in tiled preview mode, the parameter panel shows and edits that tile's parameters instead of the main shader's.
- **Speed control:** A dedicated speed slider (separate from custom params) controls `state.speed` which multiplies the time delta each frame.

SRC/RENDERER/UI/PRESETS.TS

Local per-shader parameter presets:

- **Data model:** Each grid slot has an array of `LocalPreset` objects, each containing a name and a snapshot of all parameter values at save time.
- **UI:** Preset list below the parameter sliders. Each preset has a button to recall, rename (inline edit), and delete (context menu).
- **Save:** Captures current `state.paramValues` as a new preset entry. Persists to grid state.
- **Recall:** Restores all parameter values from the preset snapshot, updating both state and UI sliders. Sends `preset-sync` to fullscreen.
- **Fullscreen sync:** Preset operations are forwarded to the fullscreen window so both renderers stay in sync.

SRC/RENDERER/UI/MIXER.TS

Multi-channel shader compositor:

- **Channel model:** `state.mixerChannels` is an array of up to 8 (`MAX_MIXER_CHANNELS`) channels. Each channel holds a `slotIndex` (which grid slot it shows), `alpha` (0.0-1.0), `blendMode` (normal, multiply, screen, add, overlay), `renderMode`, shader `code`, and `params`.
- **DOM generation:** Mixer channel DOM is created dynamically. Each channel shows: a thumbnail preview (via `MiniShaderRenderer`), an alpha slider, a blend mode dropdown, and selection highlighting for the active channel.
- **Compositing:** The mixer uses a 2D canvas overlay positioned on top of the WebGL canvas. Channels are rendered bottom-to-top:
 1. Render channel 0 to the WebGL canvas normally
 2. For channels 1+, render via `MiniShaderRenderer`, then draw onto the 2D overlay with `globalAlpha` set to the channel's alpha and `globalCompositeOperation` set to the blend mode
- **Channel assignment:** Drag-and-drop from grid slots to mixer channels. Double-click a grid slot assigns it to the active mixer channel.
- **State snapshots:** `getMixerSnapshot()` captures the full mixer state (all channels, alphas, blend modes) for visual preset save. `restoreMixerSnapshot()` restores it.

SRC/RENDERER/UI/VIEW-STATE.TS

Persistent UI layout:

- **Saved properties:** Editor panel width, preview panel height, bottom panel split position, visual presets sidebar width, panel visibility flags (editor, preview, grid, params, visual presets), last active grid tab.
- **Save:** Serializes current panel dimensions and visibility to `data/view-state.json` via IPC.

- **Restore:** `restoreViewState()` reads the saved state and applies panel sizes and visibility, including re-triggering resizer layout calculations.

SRC/RENDERER/UI/SETTINGS-DIALOG.TS

Settings overlay dialog:

- **NDI Output section:** Resolution dropdown (common presets + "Custom..." + "Match Preview"), frame skip slider (1-10). Shows current NDI status.
- **Recording section:** Resolution dropdown, codec info display.
- **Remote Control section:** Enable/disable toggle, port number, auth token (with copy button), server status display.
- **Art-Net DMX section:** Enable/disable toggle, universe number, link to Art-Net mapping dialog.
- **AI Assistant section:** Provider dropdown (Anthropic, OpenRouter), API key input, model selection, key validation.
- **Grid section:** Slot thumbnail width slider.
- **All changes:** Applied immediately and persisted via `save-settings` IPC.

SRC/RENDERER/UI/CONSOLE-PANEL.TS

Persistent message log panel below the editor:

- **Message types:** `error` (red), `warning` (yellow), `success` (green), `info` (default). Each message is timestamped.
- **DOM:** Scrollable container with automatic scroll-to-bottom on new messages. Fixed max height, collapsible.
- **API:** `consoleLog(message, type)` appends a message. `consoleClear()` empties the log.
- **Integration:** Shader compilation errors, file operation results, NDI status changes, and other system events are logged here.

SRC/RENDERER/UI/UTILS.TS

Shared UI utility functions:

- `setStatus(message, type)` : Sets the status bar text at the bottom of the window. Types: `'success'` (green), `'error'` (red), `'info'` (default). Auto-clears after 5 seconds for non-error messages.
- `updateChannelSlot(channelIndex, type, name)` : Updates the channel indicator in the toolbar (iChannel0-3 buttons) to show the current source type and name.

SRC/RENDERER/UI/CONTEXT-MENU.TS

Shared positioned context menu helper:

- **ContextMenuItem interface:** `{ label: string, action: () => void, separator?: boolean, disabled?: boolean, submenu?: ContextMenuItem[] }`.
- `showContextMenu(items, x, y, menuId?)` : Creates a positioned `<div>` with menu items at the specified screen coordinates. Supports nested submenus. Click outside or on an item dismisses the menu.
- `hideContextMenu(menuId?)` : Programmatically dismisses a specific or all context menus.
- **Used by:** Grid slots, grid tabs, visual preset slots, mix preset entries, and mixer channels for right-click actions.

SRC/RENDERER/UI/TABS.TS

Editor tab management:

- **Tab bar:** Horizontal tab bar above the editor. Each tab shows the file name (or slot label) and a close button.
- **Tab state:** Each tab stores: content (shader/scene source), cursor position, scroll position, undo stack, render mode, and associated grid slot index.
- **Tab switching:** Saves current editor state to the outgoing tab, loads the incoming tab's state into the editor. Triggers recompilation if the shader content differs.
- **Tab types:** "shader" tabs for GLSL code, "scene" tabs for Three.js JavaScript code.

SRC/RENDERER/UI/POST-PROCESS.TS

Post-processing parameter controls:

- **Parameters:** Four sliders:
 - `Luminance` (brightness adjustment, default 0.0)
 - `Hue` (hue rotation in degrees, default 0.0)
 - `Saturation` (saturation multiplier, default 1.0)
 - `Contrast` (contrast multiplier, default 1.0)
- **Export:** `ppValues` object read by renderers each frame, passed as uniforms (`iLuminance` , `iHue` , `iSaturation` , `iContrast`).
- **The actual post-processing:** Applied in the fragment shader via the uniforms — the shader itself performs the color transformations. This is not a separate render pass.

SRC/RENDERER/UI/TILING.TS

Shader repetition tiling controls:

- **Parameters:** Cols (integer), Rows (integer), Spacing (0.0-0.5), Background Color (RGB).
- **Export:** `tilingValues` object read by renderers, passed as uniforms (`iTilingCols` , `iTilingRows` , `iTilingSpacing` , `iTilingBgColor`).
- **Effect:** The shader applies the tiling by transforming UV coordinates within the fragment shader, creating a grid of repeated shader instances with optional spacing and background color between them.
- **Distinct from tiled display:** Tiling repeats the same shader within a single canvas. Tiled display renders different shaders to different regions of the fullscreen output.

SRC/RENDERER/UI/BENCHMARK.TS

GPU and CPU performance testing:

- **GPU benchmark:** Creates an offscreen `ShaderRenderer` at various resolutions, renders a stress-test shader for N frames, measures average frame time. Reports effective FPS at each resolution.
- **CPU benchmark:** Measures JavaScript overhead by timing compilation, state updates, and IPC round-trips.
- **Triggered:** From the Shader menu or via keyboard shortcut. Results displayed in a dialog.

SRC/RENDERER/UI/CLAUDE-AI.TS

AI assistant dialog for shader generation and modification:

- **UI:** Modal dialog with a text input area, send button, and streaming response display. Supports Markdown rendering of responses.
- **Prompt flow:** User types a prompt → sent via `claude-stream-prompt` IPC to Main → ClaudeManager streams response chunks → each chunk forwarded back via IPC → displayed incrementally in the dialog.
- **Code extraction:** When the AI response contains code blocks (fenced with ``` markers), an "Apply" button appears that inserts the code into the editor, replacing the current shader.
- **Attachments:** Can attach the current preview canvas as a JPEG image to the prompt, allowing the AI to see what the shader currently looks like.
- **Context:** The system prompt sent to the AI includes the current shader code, the list of available uniforms, and @param syntax documentation.

`SRC/RENDERER/UI/AB-PREVIEW.TS`

A/B crossfade preview system:

- **Model:** Two "sides" (A and B), each independently configurable:
 - **Mode:** `shader` (GLSL), `scene` (Three.js), or `composition` (mixer snapshot)
 - **Slot:** Which grid slot provides the code/params
 - **Code/Params:** Independent shader code and parameter values
- **Crossfade:** A slider (0.0 = full A, 1.0 = full B) controls the mix between sides. The compositing is done on the 2D canvas overlay using `globalAlpha`.
- **Rendering:** Each side has its own renderer instance (ShaderRenderer or MiniShaderRenderer for composition mode). Both render every frame, then are composited.
- **Fullscreen sync:** All A/B state changes (side assignment, crossfade position, mode switches) are forwarded to fullscreen via IPC through the relay.
- **Per-side tiling:** Each side can have its own tiling configuration when in composition mode.

`SRC/RENDERER/UI/ARTNET-DIALOG.TS`

Art-Net DMX mapping configuration dialog:

- **UI:** Modal overlay with a table of mappings. Each row: DMX channel (1-512), target type dropdown (param, speed, vp-recall, preset-recall, blackout, mixer-select), target name (which parameter or preset index), value range (min/max).
- **Add/Remove:** Buttons to add new mappings or remove existing ones.
- **Save:** Persists mappings to settings via IPC. Changes take effect immediately if Art-Net is active.

Grid System

`SRC/RENDERER/GRID/SHADER-GRID.TS`

Core shader grid slot management (~800 lines):

- **DOM creation:** `initShaderGrid()` creates the grid container with slot elements. Each slot has a `<canvas>` for thumbnail rendering (via MiniShaderRenderer), a label, and event listeners.
- **Slot operations:**
 - **Assign:** Load a shader into a slot (writes file to disk, updates grid state, starts thumbnail rendering)
 - **Select:** Click a slot to load its code into the editor and switch to it

- **Play**: Double-click or Enter to assign a slot to the active mixer channel (if mixer is enabled) or make it the active shader
- **Save**: Save current editor content to a slot
- **Clear**: Remove shader from a slot (deletes file, clears thumbnail)
- **Rename**: Inline label editing
- **Export/Import**: Single-slot export as `.shader` JSON file, import from `.shader` file
- **Drag-and-drop**: Drag between grid slots to reorder. Drag to mixer channels to assign. Drag `.glsl` or `.shader` files from the filesystem to import.
- **Context menu**: Right-click on a slot shows: Edit, Play, Save Here, Rename, Copy, Paste, Export, Import, Clear, Assign to Tile, Set as A/B side.
- **Tile assignment**: From context menu, assign a slot to a specific tile position in the tiled display.
- **Thumbnail rendering**: Each slot has a MiniShaderRenderer that renders 1 frame per ~100ms for a live animated thumbnail. Only visible slots are rendered (IntersectionObserver optimization from grid-renderer.ts).

SRC/RENDERER/GRID/GRID-PERSISTENCE.TS

Grid state save/load:

- **Save** (`saveGridState()`): Serializes all grid tabs with their slots to the v2 tabbed JSON format. Each slot entry contains: `index` , `name` , `type` (shader/scene/asset), `params` (current parameter values), `presets` (local presets array), `renderMode` .
- **Load** (`loadGridState()`): Reads grid state from Main via IPC, creates tab and slot DOM, compiles initial shaders for thumbnails, restores parameter values.
- **Legacy migration**: If the loaded state is the old flat-array format, wraps it in a single "Default" tab.
- **Preset import**: `importPresetsToGrid()` merges imported presets into existing grid state without overwriting other slot data.

SRC/RENDERER/GRID/GRID-TABS.TS

Grid tab bar management:

- **Tab bar**: Horizontal tabs above the grid. Each tab has a name and click handler to switch visible grid page.
- **Tab types**: `'shaders'` (default, shader code slots), `'mix'` (composition preset slots), `'assets'` (image/video asset slots).
- **CRUD**: Create new tab (with type selection), rename (inline edit), delete (with confirmation), reorder (drag).
- **Tab context menu**: Right-click on tab for: Rename, Delete, Export Tab, Import Tab.
- **Tab export/import**: Export all slots in a tab as a `.shadertab` JSON file. Import restores the tab with all its slots and shader files.

SRC/RENDERER/GRID/GRID-RENDERER.TS

Grid animation and visibility management:

- **Animation loop**: Separate `requestAnimationFrame` loop (independent from the main render loop) that updates grid slot thumbnails. Each visible slot's MiniShaderRenderer is rendered once per tick.
- **IntersectionObserver**: Only renders thumbnails for slots that are currently visible in the grid's scroll viewport. Off-screen slots are skipped entirely, saving significant GPU resources when many slots exist.

- **Container height:** Dynamically calculates and sets grid container height based on number of slots and configured slot width (`gridSlotWidth` from settings).
- **File texture caching:** When a shader has file-based channel textures, caches the loaded textures so they don't need to be reloaded when scrolling a slot back into view.

SRC/RENDERER/GRID/VISUAL-PRESETS.TS

Full scene snapshot system (Visual Presets):

- **Data model:** A visual preset captures:
 - Active shader slot index and code
 - All parameter values
 - Mixer state (all channels, alphas, blend modes)
 - Channel configurations (textures, videos, NDI sources)
 - Post-processing values
 - Tiling values
 - Render mode
 - A/B preview state (if active)
 - A thumbnail screenshot of the current output
- **Tabbed groups:** Visual presets are organized in named tabs (VPG = VP Groups). Each group contains an ordered list of presets.
- **UI:** Sidebar on the right edge of the window (collapsible/resizable). Each preset shows its thumbnail and name. Click to recall, right-click for context menu (rename, delete, move to group).
- **Recall:** Restoring a visual preset reloads the shader, applies all parameters, restores the mixer configuration, and syncs to fullscreen. This is the most comprehensive state restoration in the app.
- **Thumbnail capture:** On save, captures the current preview canvas as a JPEG thumbnail stored inline in the preset data.

SRC/RENDERER/GRID/MIX-PRESETS.TS

Mix composition presets:

- **Data model:** A composition preset captures the mixer state: number of channels, per-channel slot assignments, alphas, blend modes, and parameters. This is a subset of visual presets focused only on the mixer configuration.
- **DOM:** Displayed in a special grid tab of type `'mix'`. Each preset slot shows a label and has click/context-menu handlers.
- **CRUD:** Save current mixer state, recall (restores all mixer channels), rename, delete.
- **Context menu:** Rename, Delete, Export, Import.
- **Export/Import:** `.comp` JSON format containing the mixer snapshot.

SRC/RENDERER/GRID/ASSET-GRID.TS

Asset grid for image and video media:

- **Asset slots:** Displayed in grid tabs of type `'assets'`. Each slot shows a thumbnail of the media file and its filename.

- **Adding assets:** Drag-and-drop files onto the asset grid, or use the "Add Asset" button. Files are copied to `data/media/` via IPC.
- **Asset types:** `'asset-image'` (PNG, JPG, GIF, WebP) and `'asset-video'` (MP4, WebM, MOV).
- **Using assets:** Click an asset slot to load it into the preview via `AssetRenderer`. Drag an asset to a channel slot (iChannel0-3) to use it as an input texture.
- **Deletion:** Context menu → Delete removes the media file and the grid slot.

Tile System

`SRC/RENDERER/TILES/TILE-CONFIG.TS`

Tiled display configuration dialog:

- **UI:** Modal dialog for configuring the tiled fullscreen layout. Shows a visual grid preview where each cell can be assigned a shader slot.
- **Controls:** Rows and columns spinners (1-8 each), gap width slider, gap color picker. Per-tile assignment dropdowns listing available grid slots.
- **Preview:** Live miniature preview of the tile layout with labeled cells.
- **Apply:** Sends the configuration to the fullscreen window via IPC. Can be applied while fullscreen is running.

`SRC/RENDERER/TILES/TILE-STATE.TS`

Tile state management:

- `TileLayoutConfig` : `{ rows: number, cols: number, gapWidth: number, gapColor: string }`. Defines the physical grid layout.
- `TileData` : Per-tile data including assigned slot index, shader code, parameters, render mode, and channel textures.
- `calculateTileBounds(layout, canvasWidth, canvasHeight)` : Computes pixel-perfect bounding boxes for each tile given the layout and canvas dimensions. Accounts for gaps, rounding to integer pixels, and distributing remainder pixels evenly. Returns an array of `{ x, y, width, height }` for each tile position.
- **Tile presets:** Save/recall named tile configurations (layout + all tile assignments).
- **State persistence:** Tile layout and data saved to `data/tile-state.json` via IPC.

Fullscreen Process

`SRC/FULLSCREEN/APP.TS`

Fullscreen process entry point:

- Calls `registerIPCHandlers()` to set up communication with Main process, then calls `initFullscreen()` to initialize the rendering canvas.
- Minimal bootstrap — all logic lives in `fullscreen-renderer.ts`.

`SRC/FULLSCREEN/FULLSCREEN-RENDERER.TS`

The fullscreen output renderer (~2168 lines). Mirrors the renderer process's rendering capabilities but optimized for native-resolution output:

- **Rendering modes** (same priority as main render loop):
 1. **A/B crossfade**: Dual renderers (ShaderRenderer or scene renderer per side), composited with crossfade alpha on a 2D overlay canvas. Each side supports its own tiling configuration.
 2. **Tiled display**: Array of `TileRenderer` instances sharing a single WebGL2 context. Each tile renders to its viewport region using scissor tests. The tile grid layout is received from the renderer process.
 3. **Mixer compositing**: Array of `TileRenderer` instances (one per mixer channel), composited via 2D canvas overlay with alpha/blend modes. Same compositing approach as the main renderer's mixer.
 4. **Asset display**: Image/video rendering at fullscreen resolution.
 5. **Single shader/scene**: Standard ShaderRenderer or ThreeSceneRenderer at fullscreen resolution.
- **IPC handlers** (received from Main via relay):
 - `shader-update` — Recompiles the fullscreen shader with new source code
 - `time-sync` — Aligns the fullscreen clock with the main window
 - `param-update` — Updates parameter values for the current shader
 - `mixer-update` — Full mixer state replacement (channel count, assignments, alphas, blend modes)
 - `mixer-channel-update` — Single channel update (code, params, alpha, blend mode)
 - `mixer-remove-channel` — Removes a mixer channel
 - `tile-data-update` — Updates a specific tile's shader code and parameters
 - `tile-layout-update` — Changes the tile grid layout (rows, cols, gaps)
 - `ab-update` — Full A/B state replacement
 - `ab-set-side` — Updates one side of the A/B preview
 - `ab-set-composition` — Sets composition (mixer snapshot) on an A/B side
 - `ab-set-crossfade` — Updates the crossfade position
 - `post-process-update` — Updates post-processing values
 - `tiling-update` — Updates tiling repetition values
 - `preset-sync` — Applies a local preset from the main window
 - `set-render-mode` — Switches between shader/scene/asset
 - `asset-update` — Updates asset display (image/video path, params)
- **FPS tracking**: Measures render frame rate and periodically sends `fullscreen-fps` to Main, which forwards to the renderer for display in the stats panel.
- **Preset bar**: Keyboard shortcuts 1-9 trigger state preset recall. The preset bar is overlaid at the bottom of the fullscreen window, shown briefly on key press.
- **Channel textures**: Fullscreen maintains its own WebGL texture instances. File textures are loaded from data URLs sent via IPC. Video/camera/audio channels are not directly available in fullscreen (the main window sends rendered output frames instead). NDI input frames are forwarded from Main.

Preload Scripts

`SRC/PRELOAD/PRELOAD.TS`

Main `electronAPI` bridge exposed to the renderer process via `contextBridge.exposeInMainWorld` :

- **IPC wrappers:** Every IPC channel used by the renderer has a corresponding method on `window.electronAPI` :
 - `send(channel, ...args)` — Fire-and-forget (`ipcRenderer.send`)
 - `invoke(channel, ...args)` — Request/response (`ipcRenderer.invoke`)
 - `on(channel, callback)` — Listen for Main→Renderer messages (`ipcRenderer.on`)
- **Typed API:** Methods are named after their IPC channels:
 - File ops: `saveContent()` , `openFile()` , `newFile()`
 - Grid: `saveGridState()` , `loadGridState()`
 - NDI: `toggleNdi()` , `setNdiResolution()` , `findNdiSources()` , `setChannelNdi()`
 - Syphon: `toggleSyphon()`
 - Recording: `toggleRecording()` , `setRecordingResolution()` , `startRecording()` , `stopRecording()`
 - Remote: `toggleRemote()` , `setRemoteToken()` , `getRemoteStatus()`
 - Art-Net: `toggleArtnet()` , `getArtnetStatus()`
 - Settings: `saveSettings()` , `getSettings()`
 - AI: `claudeSetKey()` , `claudeStreamPrompt()` , `claudeListModels()`
 - Fullscreen: `openFullscreen()` , `openTiledFullscreen()` , `closeFullscreen()` , `setFullscreenDisplay()`
 - Media: `saveMedia()` , `loadMediaLibrary()` , `deleteMedia()`
 - Textures: `saveTexture()` , `loadTextures()` , `deleteTexture()`
 - Export: `exportAppState()` , `importAppState()` , `exportButtonData()` , `importButtonData()`
 - Displays: `getDisplays()`
 - Frame output: `sendFrame()`

`SRC/PRELOAD/PRELOAD-DIALOG.TS`

Preload for the custom resolution dialog window:

- `dialogAPI` : Exposes `onRequestInfo(callback)` (receives initial state from Main), `submitResult(result)` (sends user input back to Main), `cancel()` (closes the dialog).
- Used by the custom NDI/recording resolution dialog.

`SRC/PRELOAD/PRELOAD-TEXTURE-DIALOG.TS`

Preload for the texture creator dialog window:

- `textureAPI` : Exposes methods for the texture creator dialog: `onRequestInfo()` , `submitTexture(name, dataUrl)` , `cancel()` , `getExistingTextures()` .
 - Enables creating procedural textures from shader code in a sandboxed dialog window.
-

Shared Modules

Utilities

SRC/SHARED/LOGGER.TS

Configurable logging utility:

- **Log levels:** `OFF`, `ERROR`, `WARN`, `INFO`, `DEBUG`. Only messages at or above the configured level are output.
- **Logger class:** Constructor takes a name and optional level. Methods: `error()`, `warn()`, `info()`, `debug()`. Output format: `[LEVEL] [name] message`.
- `createTaggedLogger(tag)`: Factory function that creates a Logger instance with a specific tag. Used throughout the codebase: `const log = createTaggedLogger('ShaderRenderer')`.
- **Environment-based defaults:** Log level can be set via `LOG_LEVEL` environment variable. `npm run dev` sets `LOG_LEVEL=debug`.

SRC/SHARED/PARAM-PARSER.TS

Unified parser for custom shader directive comments (~500 lines):

- **@param directives:** Parses comments like `// @param name type min max default [step]`:
 - `float` — `// @param speed float 0.0 10.0 1.0`
 - `int` — `// @param count int 1 100 10`
 - `bool` — `// @param showGrid bool true`
 - `color` — `// @param bgColor color #ff0000`
 - `vec2/vec3/vec4` — `// @param offset vec2 -1.0 1.0 0.0 0.0`
- **@texture directives:** Parses `// @texture iChannelN shader(glslCode) [widthxheight]` for shader-generated input textures. Also supports `// @texture iChannelN file(path)` for file-based textures.
- **@const directives:** Parses `// @const NAME value` for compile-time constant definitions. Generates `#define NAME value` in the shader preamble.
- **@structure directives:** Parses `// @structure TypeName { field1: type1, field2: type2 }` for custom struct uniform types.
- **@option directives:** Parses `// @option name value1 value2 value3` for dropdown parameter selection.
- `parseShaderParams(source)`: Main entry point. Returns `{ params: ParamDef[], textures: TextureDirective[], constants: ConstDef[], structures: StructDef[], options: OptionDef[] }`.
- `generateUniformDeclarations(params)`: Generates GLSL `uniform` statements for all parsed parameters.
- `createDefaultParamValues(params)`: Creates a `Record<string, ParamValue>` with default values for all parameters.

Type Definitions

SRC/SHARED/TYPES/PARAMS.TS

Parameter system types:

- `ParamBaseType` : `'float' | 'int' | 'bool' | 'color' | 'vec2' | 'vec3' | 'vec4'`
- `GLSLType` : `'float' | 'int' | 'bool' | 'vec2' | 'vec3' | 'vec4'`
- `ParamValue` : `number | boolean | number[]` — Runtime parameter value
- `ParamDef` : Full parameter definition: `{ name, type, glslType, min, max, default, step, label }`
- `TextureDirective` : `{ channel: number, type: 'shader' | 'file', source: string, resolution?: string }`
- `ParamBinding` : Links a parameter to a UI control element
- `StructDef` : `{ name: string, fields: { name: string, type: string }[] }`

SRC/SHARED/TYPES/RENDERER.TS

Renderer interface and related types:

- `IRenderer` : Interface implemented by `ShaderRenderer`, `ThreeSceneRenderer`, and (partially) `TileRenderer`:
 - `compile(source: string): CompileResult`
 - `render(time: number, deltaTime: number): void`
 - `dispose(): void`
 - `setResolution(width: number, height: number): void`
 - Optional: `setChannel()`, `setParamValues()`, `getCanvas()`
- `CompileResult` : `{ success: boolean, error?: string, lineNumber?: number }`
- `RenderStats` : `{ fps: number, frameTime: number, drawCalls: number }`
- `ChannelType` : `'empty' | 'image' | 'video' | 'camera' | 'audio' | 'ndi' | 'shader'`

SRC/SHARED/TYPES/STATE.TS

Application state types:

- `RenderMode` : `'shader' | 'scene' | 'asset'`
- `SlotType` : `'shader' | 'scene' | 'asset-image' | 'asset-video' | 'empty'`
- `BlendMode` : `'normal' | 'multiply' | 'screen' | 'add' | 'overlay'`
- `TileData` : Per-tile state: `{ slotIndex, code, params, renderMode, channels }`
- `TileLayout` : `{ rows, cols, gapWidth, gapColor }`

SRC/SHARED/TYPES/SETTINGS.TS

Settings and configuration types:

- `Resolution` : `{ width: number, height: number }`
- `AppSettings` : Complete settings shape: NDI, recording, grid, remote, Art-Net, AI sections
- `SettingsDialogData` : Subset of settings exposed in the settings dialog UI
- `AISettings` : `{ provider: AIProvider, model: string, anthropicKey?: string, openRouterKey?: string }`
- `AIProvider` : `'anthropic' | 'openRouter'`
- `ClaudeModel` : `{ id: string, name: string, provider: AIProvider }`

SRC/SHARED/TYPES/PRESETS.TS

Preset data types:

- `LocalPreset` : `{ name: string, values: Record<string, ParamValue> }` — Per-shader parameter preset
- `GlobalPresets` : `{ presets: Record<string, LocalPreset[]> }` — All presets keyed by slot identifier

SRC/SHARED/TYPES/IPC-CHANNELS.TS

Typed IPC channel definitions:

- `IPCInvokeChannels` : Type map of all `ipcMain.handle` channels with their argument and return types. Used to ensure type safety between preload and main process.
- `IPCSendChannels` : Type map of all `ipcRenderer.send` channels (fire-and-forget) with argument types.
- `IPCOnChannels` : Type map of all `ipcMain.on → renderer` channels (Main-to-Renderer notifications) with payload types.

SRC/SHARED/TYPES/ARTNET.TS

Art-Net DMX types:

- `ArtNetMapping` : `{ dmxChannel: number, target: ArtNetTarget, targetName: string, min: number, max: number }`
- `ArtNetTarget` : `'param' | 'speed' | 'vp-recall' | 'preset-recall' | 'blackout' | 'mixer-select'`
- `ArtNetChange` : `{ mapping: ArtNetMapping, value: number, normalizedValue: number }` — Single DMX change event
- `ArtNetStatus` : `{ enabled: boolean, universe: number, mappingCount: number, lastPacketTime: number }`
- `ARTNET_DEFAULTS` : Default values for Art-Net configuration (universe 0, no mappings)

Data Flow & Key Patterns

Shader Compilation Flow

```
User types in editor
→ 500ms debounce (editor.ts)
→ Parse @param/@texture/@const directives (param-parser.ts)
→ Create/update ShaderTextureChannel instances for @texture directives
→ buildFragmentWrapper() wraps user code with uniform declarations (gl-utils.ts)
→ compileProgram() compiles vertex + fragment shaders (gl-utils.ts)
→ Cache all uniform locations (gl-utils.ts)
→ Update parameter UI sliders (params.ts)
→ Forward shader-update to fullscreen via relay (fullscreen-relay.ts)
→ Fullscreen recompiles independently (fullscreen-renderer.ts)
```

Frame Output Flow (NDI/Syphon/Recording)

```
Render loop completes a frame (render-loop.ts)
  → sendOutputFrames() checks skip counters (frame-sender.ts)
  → readCanvasPixels() reads GL framebuffer once (frame-sender.ts)
  → IPC send 'ndi-frame' with targets array + pixel buffer
  → Main process receives (ipc-registry.ts)
  → For each target:
    NDI: vertical flip → NDISender.send() (ndi-manager.ts)
    Syphon: vertical flip → SyphonSender.publishFrame() (syphon-manager.ts)
    Recording: vertical flip → FFmpeg stdin pipe (recording-manager.ts)
```

Remote Control Flow

```
Web client (browser)
  → HTTP request or WebSocket message with auth token
  → RemoteServer validates token (remote-server.ts)
  → For queries: RemoteServer → IPC to Main → IPC to Renderer → Renderer serializes state
  → For actions: RemoteServer → IPC to Main → IPC to Renderer → Renderer applies action
  → Statechanges: Renderer → debounced notification → Main → broadcast to all WS clients
```

Art-Net DMX Flow

```
External DMX controller sends ArtDmx packet (UDP 6454)
  → ArtNetManager receives, validates universe (artnet-manager.ts)
  → Frame diff: only changed channels emitted
  → For each change: lookup mapping, apply value or trigger
  → Batched IPC 'artnet-dmx-update' to Renderer
  → ipc-handlers.ts applies param changes / triggers preset recalls
```

Visual Preset Save/Recall Flow

```
Save:
  Capture current state → shader code, params, mixer snapshot, channels,
    post-process, tiling, render mode, A/B state (visual-presets.ts)
  → Capture preview canvas as JPEG thumbnail
  → Store in visualPresetTabs[activeTab].presets[]
  → Persist to grid-state.json
```

```
Recall:
  Load preset data → restore shader code to editor
  → Apply all param values → update UI sliders
  → Restore mixer channels (count, assignments, alphas, blend modes)
  → Restore channel textures
  → Restore post-process and tiling values
  → Forward all state to fullscreen via relay
```

Tiled Fullscreen Flow

```
Renderer: Configure tile layout via tile-config dialog
  → calculateTileBounds() computes pixel bounds (tile-state.ts)
  → Assign shader slots to tiles via grid context menu
  → IPC tile-layout-update + tile-data-update to fullscreen via relay
```

Fullscreen:

- Creates TileRenderer array, one per tile
- Each tile: compile assigned shader, set params/textures
- Render loop: for each tile, set viewport/scissor, render
- Gap areas filled with configured gap color

Feature Map

Quick reference: which files implement each major feature.

FEATURE	KEY FILES
GLSL Editing	<code>editor.ts</code> , <code>param-parser.ts</code> , <code>gl-utils.ts</code>
WebGL Rendering	<code>shader-renderer.ts</code> , <code>gl-utils.ts</code> , <code>render-loop.ts</code>
Three.js Scenes	<code>three-scene-renderer.ts</code> , <code>renderer-manager.ts</code>
Grid Thumbnails	<code>mini-shader-renderer.ts</code> , <code>grid-renderer.ts</code> , <code>shader-grid.ts</code>
Shader Grid	<code>shader-grid.ts</code> , <code>grid-persistence.ts</code> , <code>grid-tabs.ts</code> , <code>grid-renderer.ts</code>
Custom Parameters	<code>param-parser.ts</code> , <code>params.ts</code> , <code>shader-renderer.ts</code>
Local Presets	<code>presets.ts</code> , <code>grid-persistence.ts</code>
Visual Presets	<code>visual-presets.ts</code> , <code>grid-persistence.ts</code>
Mix Presets	<code>mix-presets.ts</code> , <code>grid-tabs.ts</code>
Mixer Compositing	<code>mixer.ts</code> , <code>render-loop.ts</code> , <code>mini-shader-renderer.ts</code>
Tiled Display	<code>tile-config.ts</code> , <code>tile-state.ts</code> , <code>tile-renderer.ts</code> , <code>fullscreen-renderer.ts</code>
A/B Crossfade	<code>ab-preview.ts</code> , <code>render-loop.ts</code> , <code>fullscreen-renderer.ts</code>
Post-Processing	<code>post-process.ts</code> , <code>shader-renderer.ts</code> , <code>gl-utils.ts</code>
Tiling Repetition	<code>tiling.ts</code> , <code>shader-renderer.ts</code> , <code>gl-utils.ts</code>
Asset Media	<code>asset-renderer.ts</code> , <code>asset-grid.ts</code> , <code>grid-tabs.ts</code>
Channel Textures	<code>shader-renderer.ts</code> , <code>shader-texture-channel.ts</code> , <code>ipc-handlers.ts</code>
NDI Output	<code>ndi-manager.ts</code> , <code>ndi-sender.ts</code> , <code>frame-sender.ts</code>
NDI Input	<code>ndi-manager.ts</code> , <code>ndi-receiver.ts</code> , <code>ipc-handlers.ts</code>
Syphon Output	<code>syphon-manager.ts</code> , <code>syphon-sender.ts</code> , <code>frame-sender.ts</code>
Recording	<code>recording-manager.ts</code> , <code>frame-sender.ts</code>
Fullscreen Output	<code>window-manager.ts</code> , <code>fullscreen-relay.ts</code> , <code>fullscreen-renderer.ts</code>
Remote Control	<code>remote-manager.ts</code> , <code>remote-server.ts</code> , <code>ipc-handlers.ts</code>
AI Assistant	<code>claude-manager.ts</code> , <code>claude-ai.ts</code>
Art-Net DMX	<code>artnet-manager.ts</code> , <code>artnet-dialog.ts</code> , <code>ipc-handlers.ts</code>

FEATURE	KEY FILES
Settings	<code>settings-manager.ts</code> , <code>settings-dialog.ts</code>
State Persistence	<code>file-manager.ts</code> , <code>grid-persistence.ts</code> , <code>view-state.ts</code> , <code>tile-state.ts</code>
App Export/Import	<code>export-manager.ts</code>
Menus	<code>menu-builder.ts</code> , <code>context-menu.ts</code>
Beat Detection	<code>beat-detector.ts</code> , <code>shader-renderer.ts</code>
Benchmark	<code>benchmark.ts</code>
Console Log	<code>console-panel.ts</code>
IPC Bridge	<code>preload.ts</code> , <code>ipc-registry.ts</code> , <code>ipc-handlers.ts</code> , <code>fullscreen-relay.ts</code>
Logging	<code>logger.ts</code>
Type Safety	<code>types/params.ts</code> , <code>types/renderer.ts</code> , <code>types/state.ts</code> , <code>types/settings.ts</code> , <code>types/presets.ts</code> , <code>types/ipc-channels.ts</code> , <code>types/artnet.ts</code>